Praxis der Softwareentwicklung

Visualisierung von Verifikationsalgorithmen für Neuronale Netze

Praxis der Softwareentwicklung (PSE)
Wintersemester 2025/26

Implementierungsbericht

Karlsruher Institut für Technologie (KIT)
KASTEL-Institut für Informationssicherheit und Verlässlichkeit
Anwendungsorientierte Formale Verifikation
Prof. Dr. Bernhard Beckert
Am Fasanengarten 5
Gebäude 50.34
76131 Karlsruhe

Betreuer: Philipp Kern

Projektteilnehmer:

Name	E-Mail-Adresse
Alexander Mikhaylov	uiqbn@student.kit.edu
Cedric Linde	unogx@student.kit.edu
Elias Dörr	uuxmf@student.kit.edu
Enrique Lopez	ukgyh@student.kit.edu
Paul Schepperle	ucnlo@student.kit.edu

Karlsruhe, 18. März 2026

Inhaltsverzeichnis

1 Einleitung **3**

2 Umsetzung **4**
2.1 Kriterien 4
2.1.1 Muss-Kritieren 4
2.1.2 Soll-Kriterien 5
2.1.3 Kann-Kriterien 5
2.1.4 Nichtfunktionale Anforderungen 5
2.2 Umsetzung der Design-Patterns 6
2.3 Statistiken zum Ablauf 7
2.4 Fazit zum Ablauf 8

3 Änderungen Entwurfsheft **9**
3.1 Neue Klassen 9
3.2 Entfernte Klassen 11
3.3 User Interface 11

4 Zeitplan **14**
4.1 Gantt-Diagramm 14

Glossary **16**

1 Einleitung

In dieser Phase der Projektentwicklung wurden die ausführlichen Anforderungen, Definitionen und UI-Skizzen genutzt, um die erste Version des eigentlichen Programms mit Python zu Implementieren. Dazu haben wir im Entwurf schon einen Zeitplan entworfen, um alle Features umsetzen zu können und Zeitdruck vorzubeugen. In diesem Heft blicken wir auf die Erfolge und Probleme der Implementierungsphase zurück und bewerten, ob diese ein Erfolg war. Außerdem gehen wir genau auf alle Änderungen und Abweichungen der vorherigen Plaungsphasen ein.

Die grundsätzliche Idee des Programms ist es, mithilfe schon existierender Verfikationsalgorithmen die erreichbaren Wertemengen beliebiger Neuronenpaare eines Netzes zu visualisieren. Die bekannten Algorithmen können dies bereits für die Knoten des Ausgabelayers. Unser Programm besitzt nun die Möglichkeit, Netze so zu modifizieren, dass nur ausgewählte Knoten Auswirkungen auf die Ausgabeschicht haben. Somit berechnen die Algorithmen effektiv die erreichbaren Werte dieser ausgewählten Knoten. Fügt man nun zusätzliche Endknoten ein, welche mit ihren Kantengewichten neue Linearkombinationen der ausgewählten Neuronen repräsentieren, so lässt sich die erreichbare Wertemenge noch präziser bestimmen.

2 Umsetzung

2.1 Kriterien

Im Pflichtenheft ist eine Liste an Anforderungskriterien definiert und in drei verschiedene Prioritäten aufgeteilt. Im Fogenden wird überprüft, welche dieser Kriterien tatsächlich implementiert werden konnten.

2.1.1 Muss-Kritieren

Es konnten alle definierten Muss-Kriterien umgesetzt werden.
Das Programm muss...

⟨KM1⟩ verschiedene Netze und zugehörige Informationen einlesen können.

⟨KM2⟩ aus 2 Ansichten bestehen. Es gibt eine Ansicht, die die Netze darstellt, und eine Ansicht, die die Diagramme darstellt.

⟨KM3⟩ die Möglichkeit, über ein Interface neue Algorithmen in das Programm zu implementieren, bieten.

⟨KM4⟩ eine Auswahl von 2 Neuronen aus einem Netz unterstützen.

⟨KM5⟩ einen dynamischen Wechsel von Netzen, Verifikationsalgorithmen, den zu untersuchenden Neuronen und dem Eingabeset unterstützen.

⟨KM6⟩ die Darstellung von erreichbaren Mengen von Eingabemengen an spezifischen Neuronen in einem 2D Graphen unterstützen.

⟨KM7⟩ die Graphen als .png exportieren können.

⟨KM8⟩ eine automatische Erkennung von stark aktivierten Neuronen bei Durchläufen von Eingaben unterstützen.

⟨KM9⟩ Aktionen und Netze aus letzten Sessions speichern.

2.1.2 Soll-Kriterien

Zumindest eins der beiden Soll-Kriterien konnte umgesetzt werden. Das andere Kriterium ⟨KS1⟩ (Visualisierung von 3 Neuronen als 3D-Plot) wurde aufgrund von unverhältnismäßiger Komplexität übersprungen.
Das Programm soll...

⟨KS2⟩ die Graphen als .svg-Dateien exportieren können.

2.1.3 Kann-Kriterien

Auch von den Kann-Kriterien konnte zumindest ein Kriterium umgesetzt werden. Da die Anpassbarkeit des UIs ⟨KK2⟩ für die Benutzbarkeit des Programms keine große Relevanz hat, haben wir das Kriterium übersprugen. Für die Anpassung der Kantengewichte ⟨KK4⟩ wäre die Nutzung der großen Bibliothek PyTorch nötig gewesen. Da wir diese ansonsten nicht benutzen, haben wir uns dazu entschieden, diese Funktion nicht umzusetzen.
Das Programm kann...

⟨KK5⟩ die Gewichte von Kanten visuell darstellen.

2.1.4 Nichtfunktionale Anforderungen

Wie geplannt läuft das Programm komplett offline. Das Programm hat ein konsistentes Design mit Farbpaletten für beide Hauptansichten. Das Programm führt genau die vom Nutzer ausgewählten Algorithmen aus. Die durch das Programm erhobenen Produktdaten werden weder intern noch an Dritte weitergegeben. Außerhalb der Produktdaten erfolgt keine Datenerhebung. Außerdem wurde eine geeignete Open-Source-Lizenz gewählt.

Damit wurden alle nichtfunktionalen Sicherheits-, Datenschutz, Urheber- und Markenrechtsanforderungen erfüllt.

2.2 Umsetzung der Design-Patterns

- **Model-View-Controller**: Für alle drei Komponenten wurde ein Package im Projekt angelegt, um eine übersichtliche Trennung zu ermöglichen. Auch die Utils-Komponente hat ein eigens Package erhalten.
- **Singletons**: Singletons wurde mithilfe der Meta-Klasse `Singleton` in `Utils` implementiert. Alle Konstruktor-Aufrufe der Child-Klassen von `Singleton` werden Abgefangen und nur beim aller ersten Aufruf eine Instanz pro Child-Klasse erstellt und gespeichert. Bei allen späteren Aufrufen wird die gespeicherte Instanz zurückgegeben, anstatt eine neue Instanz zu erstellen. Zum Benutzen eines Singletons muss also lediglich der Konstruktor der Klasse aufgerufen werden, sodass der Nutzer der Klasse nichts von der Singleton-Logik mitbekommt.
- **Observer für Algorithmen-Dateien**: Um Updates des Dateisystems zu erhalten und dementsprechend die Algorithmen zu aktuallisieren haben wir die neue Klasse `AlgorithmFileObserver` angelegt. Diese nutzt die Bibliothek watchdog und erbt von dessen `FileSystemEventHandler`, um bei neuen Events benachrichtigt zu werden.
- **Strategy zum Ausführen von Algorithmen**: Das im Entwurf definierte Interface wurde wie geplant umgesetzt, um beliebige vom User erstellte Algorithmen auf den Netzen auszuführen. Es ist erforderlich, dass die bestimmte Methode mit dem Namen `calculate_output_bounds` in der .py Datei vorhanden ist, die genau 2 Parameter (onnx-Netzwerk und Input-Bounds) akzeptiert. Die Methode wird beim Laden von `AlgorithmLoader` im Cache gespeichert. `AlgorithmExecutor` holt die Funktion aus Cache, die Berechnungen werden durchgeführt und als Output bekommt man die Output-Bounds.

2.3 Statistiken zum Ablauf

Insgesamt besteht unser Programm aus:

- 6378 Codezeilen
- 629 Kommentarzeilen

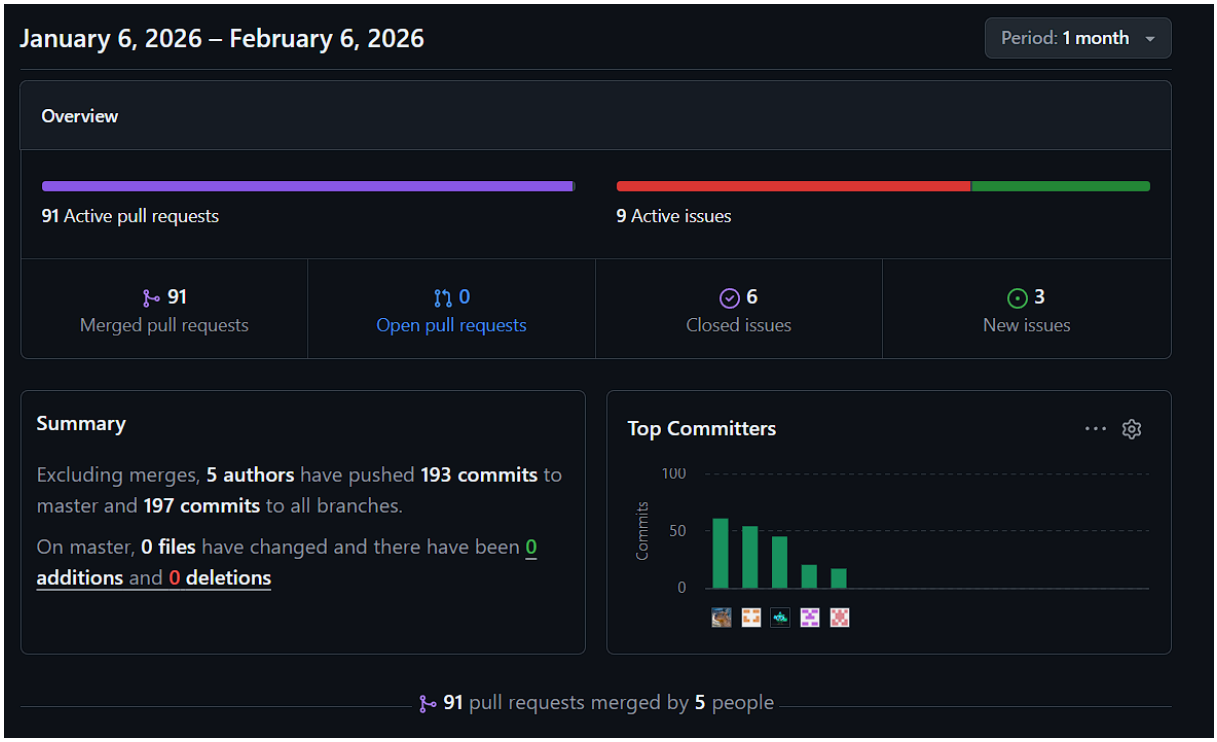

Abbildung 2.1: Github Statistiken

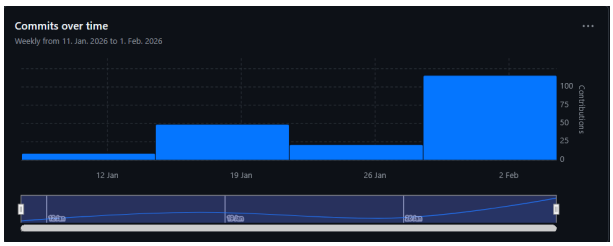

Abbildung 2.2: Überblick über Commit-Entwicklung

2.4 Fazit zum Ablauf

Die Implementierungsphase hat als Ergebnis ein Programm, welches funktioniert und alle Muss-Kriterien erfüllt. Es mussten zwar einige Kann- und Soll-Kriterien aufgrund von Zeitdruck gestrichen werden, die restlichen Aufgaben konnten jedoch gut verteilt und gemeinsam erledigt werden. Die Kommunikation zwischen den einzelnen Features und Komponenten funktioniert gut.
Abschließend kann die Implementierungsphase als Erfolg bewertet werden.

3 Änderungen Entwurfsheft

3.1 Neue Klassen

- `SampleMetric` ist eine Datenklasse für eine einzelne Auswertungsregel bei der Sample-Analyse. Sie definiert einen eindeutigen Schlüssel, einen Anzeigenamen und eine Funktion, die aus den Ausgabewerten der Stichproben (ein numpy-Array mit Form num_samples × output_größe) eine Kennzahl pro Output-Neuron berechnet, z.,B. Maximum oder Mittelwert. Diese Metriken werden im „Run Samples“-Dialog auswählbar gemacht und bestimmen, welche Kennzahlen im Ergebnis erscheinen
- `AlgorithmFileObserver` wurde aus `Storage` ausgelagert und implementiert das kontinuierliche Prüfen nach Änderungen im Algorithmen-Ordner. Diese Funktionalität ermöglicht es dem Nutzer, selbst Algorithmen zu dem Programm hinzuzufügen und mit diesen Vergleiche zu erstellen. Dafür existiert in `Storage` eine Liste an Algorithmen, die durch `AlgorithmFileObserver` aktuell gehalten wird. Das Auslagern kam dadurch zustande, dass die Directory-watching Funktionalität mit der Zeit immer komplexer wurde und eine unabhängige Funktion darstellt.
- `Singleton` ist eine Meta-Klasse, die jede Child-Klasse in einen Singleton konvertiert. Da unser Programm viele Singletons nutzt und so nicht jedes mal die Singleton-Logik erneut implementiert werden muss, konnten wir uns mit der Klasse einige Zeilen Code und sparen. Meta-Klassen in Python haben die Möglichkeit, Konstruktor-Calls ihrer Child-Klassen abzufangen und zu verhindern, dass jedes mal eine neue Instanz von diesen erstellt wird. Das machen wir uns zunutze, um nur beim ersten Aufruf pro Klasse eine neue Instanz zu erstellen, diese in `_instances` zu speichern und bei allen weiteren Aufrufen zurückzugeben.
- `ColorManager` verwaltet das Farbschema des Programms. Es beinhaltet jeweils eine Liste an Farben mit zugeordneten Namen für das blaue und das grüne Farbschema und eine Funktion, um eines der beiden Schemata zu laden. Dazu läd es das Stylesheet aus der `style.qss`-Datei, in welchem alle vom Farbschema abhängigen Farben mit `@name` eingetragen sind. Dann ersetzt es diese labels durch die eigentlichen Farben und wendet das Stylesheet auf das granze Programm an.

- `NetworkEdgeBatch` wurde eingeführt, um das Rendern der Verbindungen eines Netzwerks performanter zu machen. In pyside6 werden eigenhändig auf den Screen gezeichnete Objekte jeweils durch `QGraphicsItem`s repräsentiert. Diese Basisklasse implementiert grundlegende Funktionalität wie die Interaktion mit dem Nutzer und Kollisionserkennung. Da jedoch (im Gegensatz zu den Knoten) die Linien in einem Netzwerk nicht interaktiv sind, ist es nicht nötig, all diese zusätzlichen Möglichkeiten zu unterstützen. Darüber hinaus führte werden pro Netz teilweise hunderte bis tausende Kanten gezeichnet, weshalb sich kleine Performance-Einschränkungen durch eigene Objekte pro Kante sehr schnell zu einer stotternden Nutzeroberfläche aufaddieren. Zudem können durch das Verarbeiten aller Kanten auf einmal die Parameter zum Zeichnen dieser besser kontrolliert werden. Dies war auch die Grundlage dafür, dass für noch größere Netze die Kanten einer Schicht gesammelt als ein Polygon gezeichnet werden können. Die Kanten zusammen in einem `QGraphicsItem` zu repräsentieren bringt also diverse Vorteile, ohne die Funktionalität des Programms einzuschränken.
- `NetworkLayerLine` ist eine weitere Performance-Optimierung des `NetworkWidget`s. Sie wurde eingeführt, da bei sehr großen Netzen (deren Kanten durch Polygone dargestellt werden) die Anzahl der Knoten irgendwann so groß ist, dass das Rendern dieser signifikanten Zeitaufwand mit sich bringt. Zusätzlich ist die normale Ansicht bei großen Netzen derart herausgezoomt, dass einzelne Knoten kaum voneinander zu unterscheiden sind. Aus diesen Gründen wird für große Netze zusätzlich zu den einzelnen Knoten eine `NetworkLayerLine` gezeichnet, die den visuellen Eindruck einer Knotenschicht verdeutlicht. Um dabei Rechenaufwand zu sparen, werden die Knoten bei einem ausreichend kleinen Zoom-Grad nicht mehr angezeigt. Die einzelnen Knoten müssen aber prinzipiell trotzdem als Objekte existieren, da die Funktionalität erhalten bleiben soll, dass der Nutzer einzelne Neuronen im Netz auswählen kann.
- `BoundsDisplayWidget` ist ein UI-Widget zur Anzeige eines Bounds-Satzes. Es erzeugt für jede Eingabedimension eine Zeile mit Index, Minimum und Maximum. Es wird z.,B. im Netzwerk-Sidebar genutzt, um die aktuell ausgewählten gespeicherten Bounds lesbar anzuzeigen.
- `SampleMetricsWidget` visualisiert die Ergebnisse der Sample-Berechnung. Es liest das Ergebnis-Dict, das beim „Run Samples“ erzeugt und im jeweiligen Bounds-Objekt gespeichert wird, und stellt eine Zusammenfassung (Anzahl Samples, auswählbare Kennzahl) sowie die wichtigsten Aktivierungen dar. Optional zeigt es detaillierte Werte pro Output/Layers, sortiert nach Kennzahl, mit Begrenzung der Anzeige für große Outputs. Es wird sowohl im Network-Sidebar als auch im Neuron-Picker genutzt, um gespeicherte Sample-Ergebnisse schnell zu überprüfen.
- `PlotSettingsWidget` repräsentiert die Einstellungs-Karte, die links in der Seitenleiste des PlotViews für jeden Plot angezeigt wird. Da sie für jeden Plot einzeln mehrere Variablen

speichert und individual aktuallsiert werden muss, haben wir uns dazu entschieden, sie in ein eigenenes Widget auszulagern.

3.2 Entfernte Klassen

- `ActionMenuItem` wurde entfernt, da jedes Item im action menu nur ein einfacher button ist und außerdem nur ein einziges `ActionMenu` existiert. Somit war die Klasse überflüssig.
- `SideBar` hat keine besonderen Eigenschaften oder einen eigenen Style. Deshalb wurde die Klasse entfernt und stattdessen in `Tab` ein beliebiges `QWidget` als Sidebar akzeptiert.
- `ZoomPanel` ist nur ein kleines UI-Element, welches nur Funktionalität hinzufügt, welche durch Maus- und Touchpadgesten bereits existiert. Deshalb wurde es fürs Erste entfernt. Um die Nutzbarkeit und Barrierefreiheit des Programms zu erhöhen ist eine spätere Implementierung jedoch nicht ausgeschlossen.
- `CloseDialogButton` wurde nur von `DialogBase` genutzt und ist außerdem ein einfacher icon-button. Da das einheitliche Design aller Icons klassenunabhängig im Stylesheet verwaltet wird, gab es für die Klasse keine Daseinsberechtigung.
- `FullscreenPlotDialog` war ungenutzt, da der Vollbildmodus eines Plots schon in MatPlotLib integriert war und bei Aktivierung ein eigenes Fenster öffnet, anstatt unser eigenes Dialogsystem zu nutzen.
- `PlotLoader` und `PlotExporter` wurden entfernt, da sie für die Logik von SaveState überflüssig sind. `SaveStateLoader`, `SaveStateExporter` und `Storage` übernemmen die ganze Logik und weitere Klassen sind nutzlos.
- `ImageExporter` war ungenutzt, da der Export eines Bildes schon in MatPlotLib integriert war und es das Plotbild als PNG oder mehrere weitere Optionen exportieren lässt.
- `PlotConfigDialogController` war ungenutzt, da die gesamte Logik von PlotViewController übernommen wurde.

3.3 User Interface

Auch am UI haben sich aus verschiedenen Gründen einige Elemente verändert.

Drop-Shadows:
Aufgrund der Limitationen von PySide6 haben wir uns dazu entschieden, alle Drop-Shadows aus dem Design zu entfernen. Schatten sind in QT zwar grundsätzlich möglich, ihre Implementierung ist jedoch sehr umständlich. Außerdem können sie im Gegensatz zu klassischem CSS nicht im

QSS-Stylesheet gesetzt werden, sondern müssen für jedes Widget einzeln im Python-Code kreiert werden.

Tutorial Speech Bubble:
Die `TutorialSpeechBubble` hat ein neues, schichteres Design bekommen, welches keine Sprechblase mehr darstellt. Dafür gibt es zwei Gründe. Zum einen wird der leere TabView so besser ausgefüllt, wodurch das Programm beim Starten weniger leer aussieht. Zum anderen hat eine verständliche Erklärung des Programms zu viel Platz für eine kleine Sprechblase benötigt.

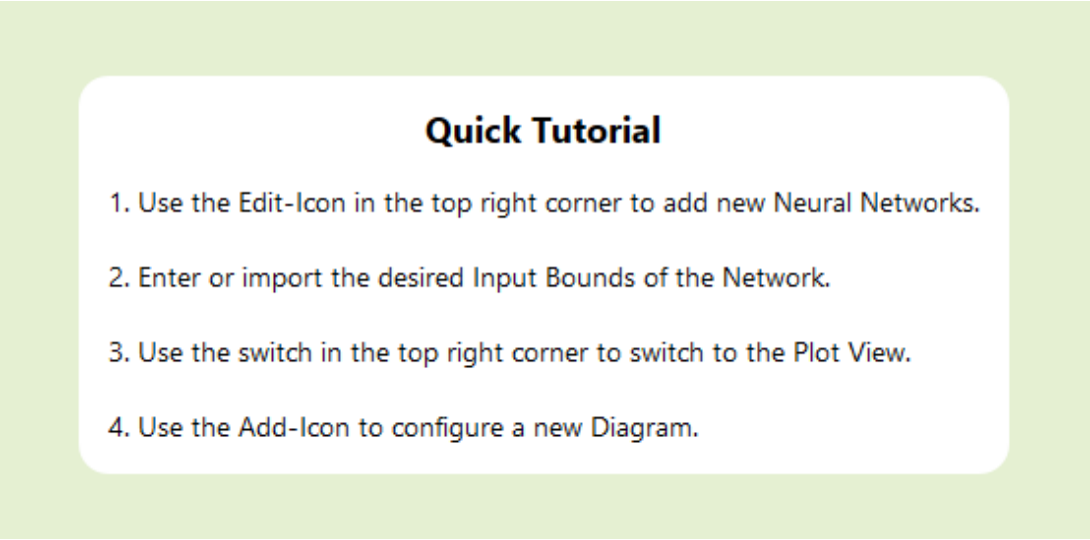

Abbildung 3.1: Tutorial Speech Bubble

PlotWidget Das `PlotWidget` hat ebenfalls ein neues Design bekommen. Der Grund dafür ist, dass die obere Button-Reihe automatisch von MatPlotLib erstellt wurde. Somit konnten wir uns zwar etwas Arbeit sparen und den Funktionsumfang des Widgets sogar erhöhen, waren dafür aber im Styling der Buttons limitiert.

RunSamplesDialog Für den Dialog zum Erstellen und Ausführen von Testdaten hatten wir in den vorherigen Phasen noch kein Design definiert.

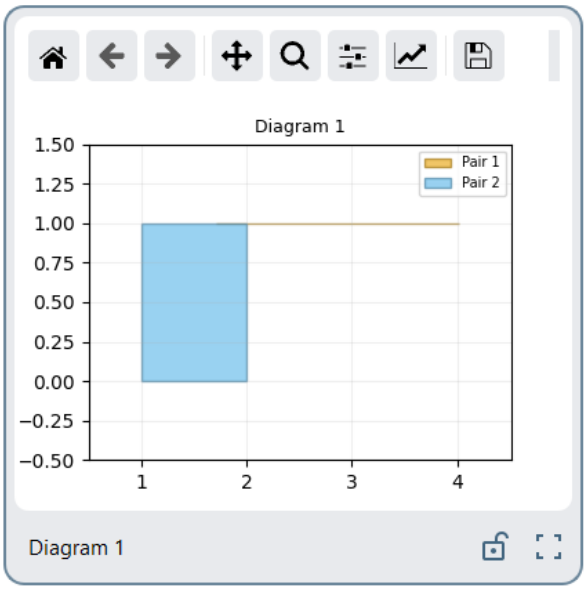

Abbildung 3.2: Beispiel für ein Plot Widget

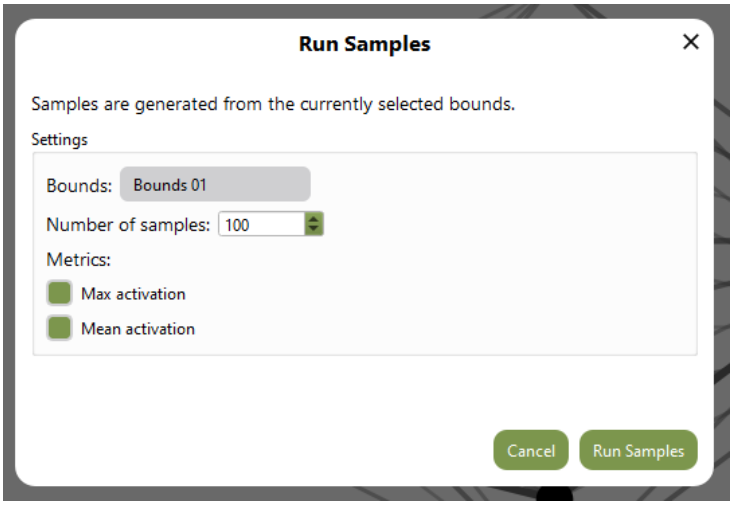

Abbildung 3.3: Run Sample Dialog

4 Zeitplan

Unser Programm besteht aus verschiedenen funktionalen Anforderungen, die zum Großteil unabhängig voneinander Implementiert werden konnten. Viele von ihnen haben allerdings ein Mindestmaß an Struktur vorausgesetzt. Deshalb haben wir die ersten Tage der Implementierungsphase genutzt, um möglichst schnell alle Klassen und Dateien anzulegen, das Fenster des Programmes zu öffnen und den Wechsel zwischen den beiden Hauptansichten zu ermöglichen.
Danach konnten wir alle unabhängig voneinander verschiedene Funktionen Implementieren, die von dem Import der verschiedenen Datentypen über die Visualisierung von Neuronalen Netzen bis hin zum Erstellen neuer Plots und UI-Design gereicht haben. Viele dieser Funktionen haben zu diesem Zeitpunkt nur Beispiel-Daten genutzt und konnten so trotzdem designed und getestet werden.
Es konnten zwar viele Features einzeln implementiert werden, damit das Programm funktioniert müssen allerdings schlussendlich fast alle Funktionen miteinander interagieren. Deshalb hat es am Ende der Implementierung einige Tage gedauert, alle Features miteinander zu verbinden und Daten zwischen ihnen zu übergeben. Vor allem am Ende war daher gute Kommunikation zwischen allen Teammitgliedern gefragt, da viele Schnittstellen miteinander abgestimmt werden mussten.

4.1 Gantt-Diagramm

Beim Vergleich mit dem Gantt-Diagramm aus dem Entwurfsheft fällt auf, dass sich einige Aufgaben weiter nach hinten verschoben haben und dadurch der Arbeitsaufwand in der zweiten Hälfte deutlich höher als zunächst erwartet war. Das hängt vor allem damit zusammen, das der Entwurf einige Tage länger als erwartet gedauert hat, weswegen wir in der ersten Woche der Implementierungsphase nur kleinere Aufgaben erledigen konnten. Auch die eingeplante Pufferzeit wurde dadurch aufgebraucht.
Der Zeitaufwand der einzelnen Aufgaben war größtenteils wie erwartet.

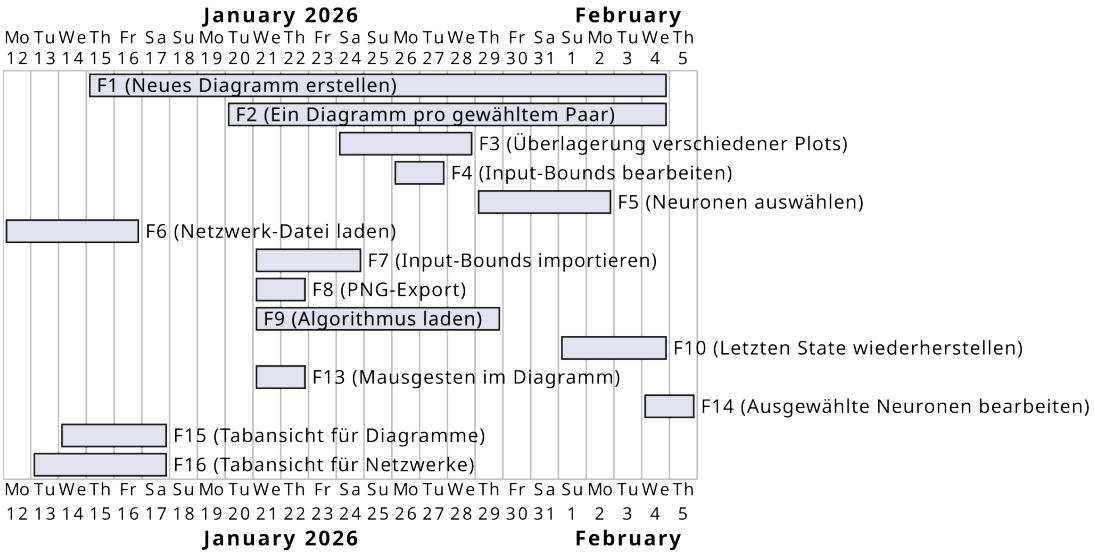

Abbildung 4.1: Gantt Diagramm

Glossar

.csv Dateiformat zum Verwalten von Text- und Zahlenwerten in Form von Tabellen. Wird im Programm für den Import und Export der Bounds eines Netzes benutzt.

.onnx Open Neural Network Exchange, quelloffenes Dateiformat zum Speichern von Neuronalen Netzen. Kann zum Auswerten importiert werden..

.py Dateiformat eines Pythonskriptes. Kann genutzt werden, um eigene Algorithmen in die Software einzubinden.

.svg Dateiformat für Vektorgrafiken. Bilder werden nicht als als einzelne Pixel gespeichert, sondern als mathematisch definierte Objekte wie Linien, Kreise und Kurven.

.vnnlib Beinhaltet die Input- und Output Bounds eines Netzwerks, nicht aber das Netzwerk selbst.

Bibliothek Ansammlung an Softwarecode anderer Entwickler, die diesen als Paket zur Einbindung in fremde Projekte zur Verfügung stellen..

Binding Verbindung zwischen zwei Softwarekomponenten, bspw. zwischen dem Programm und einer in einer anderen Sprache geschriebene Bibliothek..

Caching Die Speicherung zuvor geladener oder berechneter Daten im Zwischenspeicher, ermöglicht einen schnellen erneuten Zugriff auf diese.

Code Coverage Tool Tool zur Erfassung davon, wie viel des Codes durch Tests abgedeckt ist.

deterministischer Algorithmus Algorithmus, der bei gleicher Eingabe immer zum gleichen Ergebnis kommt.

Diagramm Stellt den Zusammenhang eines Neuronenpaares mit einem Algorithmus dar. Kann entweder als einzelner Tab oder als Teil eines Vergleiches angezeigt werden.

draggen Bewegen der Maus während ein Button gedrückt gehalten wird. Geste, um UI-Elemente zu verschieben oder im dreidimensionalen Raum zu drehen..

Dropdown-Menü Ein UI-Element zum Auswählen eines Entrags aus einer vorgegebenen Liste an Optionen, wird zur Auswahl von Neuronalen Netzen und Algorithmen benutzt.

einsatzbereites Netz Ein Netz, bei dem Eingabebounds schon geladen/eingestellt wurden.

großes Netz Ein Netz mit mindestens 100000 Knoten.

GUI Graphical User Interface - Die Benutzeroberfläche des Programms.

Layer Einzelne Schicht eines neuronalen Netzes. Es gibt einen Input- und einen Outputlayer für die Ein- und Ausgabe des Netzes.

Log-Datei Datei zur Speicherung von Systemnachrichten.

MemoryError Ein speicherbedingter Fehler.

Neuron Ein einzelner Knoten eines Netzwerkes. Wird über gewichtete Kanten mit anderen Neuronen verbunden.

Neuronales Netz Netz aus künstlichen Neuronen, an biologischen Gehirnen inspiriert. Besteht aus Nodes in Layern und gewichteten Verbindungen.

Open-Source-Lizenz Eine Lizenz für quelloffene Software.

pan Bewegen eines UI-Elements durch Draggen der Maus in zwei Dimensionen. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.

PEP 8 Ein Style Guide für Python Code.

Polytop Eine Verallgemeinerung von konvexen Vielecken in beliebiger Dimension.

Session Engl. für Sitzung, beschreibt die Zeitperiode ab dem Öffnen des Programms durch einen Nutzer bis zum vollständigen Schließen..

Standartnetz Ein Netz mit 2000 Neuronen, die auf 4 Layer verteilt sind.

System Usability Scale Tool zur Erfassung der subjektiv empfundenen Usability.

Tab Ein "karteikartenartiger" Reiter in einer Auflistung, wird zum Wechseln der Ansicht zwischen verschiedenen Neuronalen Netzen, Diagrammen und Vergleichen verwendet.

Unit Test Test zur Überprüfung eines klar abgegrenzten Teils des Programms.

Vergleich Ein Vergleich zeigt Diagramme mehrerer Neuronenpaare.

Zonotop Ein Zonotop ist die Minkowski-Summe einer endlichen Anzahl an Vektoren. Die enstandene Fläche ist konvex. Es wird im Diagramm dargestellt und aus neuronalen Netzen generiert.

zoom Ändern der Anzeigegröße eines UI-Elements. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.